

Atividade Teórica e Prática 1

SISTEMAS DISTRIBUÍDOS

NOME: FELIPE ALEXANDRE VIEIRA MENDES MATRICULA: 0030839	DATA: 09/08/2026
--	-------------------------

Além de resolver a lista de exercícios 1 disponível na pasta da disciplina, você deverá demonstrar todos os tipos de formatos de serialização baseados em texto:

- CSV
- JSON
- XML
- YAML
- TOML

Para isso, e utilizando uma linguagem de sua escolha, desenvolva um pequeno sistema de troca de mensagens entre 1 cliente e 1 servidor via sockets. O sistema deve permitir o cliente enviar as seguintes informações em uma única mensagem ao servidor:

- Nome
- CPF
- idade
- mensagem

Utilize os formatos de serialização para organizar as informações e seus respectivos “valores”. Exemplo: Nome: Fulano, CPF: 10326709722, idade: 45, mensagem: segue comprovante de endereço. Considerando que são 5 formatos, o cliente deve enviar 5 vezes, e o servidor receber, manipular e imprimir as informações 5 vezes.

1. Introdução

O objetivo desta atividade foi desenvolver uma aplicação simples de comunicação entre um cliente e um servidor em um sistema distribuído, utilizando sockets TCP e diferentes formatos de serialização baseados em texto.

A aplicação troca uma mesma estrutura de dados utilizando CSV, JSON, XML, YAML e TOML. Dessa forma, foi possível observar que a informação pode ser representada de maneiras diferentes, mesmo mantendo os mesmos valores.

2. Tecnologias utilizadas

Foi utilizada a linguagem Python por possuir suporte multiplataforma e uma biblioteca padrão com recursos para sockets, CSV, JSON e XML.

Para YAML foi utilizada a biblioteca PyYAML.

Biblioteca/módulo	Utilização
socket	Comunicação TCP
csv	CSV
json	JSON
xml.etree.ElementTree	XML
yaml / PyYAML	YAML
serializadores/toml_serializador.py	TOML

3. Arquitetura

O sistema possui:

- `cliente.py`: cria, serializa e envia.
- `servidor.py`: recebe, desserializa e imprime.
- `modelo.py`: representa a mensagem.
- `protocolo.py`: controla o tamanho das mensagens TCP.
- `serializadores/`: possui uma biblioteca/módulo separado para cada formato.

4. Comunicação por sockets

Foi utilizado TCP para realizar a comunicação confiável entre os processos.

O cliente inicia a conexão e envia cinco mensagens consecutivas. O servidor recebe cada mensagem, aplica o desserializador correspondente e recupera os quatro campos originais.

5. Controle do tamanho

TCP é um fluxo contínuo de bytes. Portanto, uma chamada de ``recv()`` não deve ser tratada como se representasse necessariamente uma mensagem inteira.

O projeto utiliza um cabeçalho de 4 bytes antes de cada conteúdo. Esse cabeçalho informa o tamanho da mensagem.

O servidor primeiro lê o tamanho e depois chama uma função que continua recebendo até obter todos os bytes da mensagem.

6. Dados utilizados

Nome: Fulano da Silva

CPF: 10326709722

Idade: 45

Mensagem: segue comprovante de endereço

O CPF foi tratado como texto porque é um identificador e não um valor usado para cálculo.

7. Formatos

CSV

nome,cpf,idade,mensagem

Fulano da Silva,10326709722,45,segue comprovante de endereço

Arquivo:

serializadores/csv_serializador.py

JSON

```
{  
  "nome": "Fulano da Silva",  
  "cpf": "10326709722",  
  "idade": 45,  
  "mensagem": "segue comprovante de endereço"  
}
```

Arquivo:

serializadores/json_serializador.py

XML

```
<mensagem>  
  <nome>Fulano da Silva</nome>  
  <cpf>10326709722</cpf>  
  <idade>45</idade>  
  <mensagem>segue comprovante de endereço</mensagem>  
</mensagem>
```

Arquivo:

serializadores/xml_serializador.py

YAML

```
nome: Fulano da Silva  
cpf: '10326709722'  
idade: 45  
mensagem: segue comprovante de endereço
```

Arquivo:

serializadores/yaml_serializador.py

TOML

```
nome = "Fulano da Silva"  
cpf = "10326709722"  
idade = 45  
mensagem = "segue comprovante de endereço"
```

Arquivo:

serializadores/toml_serializador.py

8. Funcionamento do cliente

O cliente cria um objeto `Mensagem` e monta uma lista contendo os cinco serializadores.

Para cada formato, ele:

1. serializa os dados;
2. mostra a representação no terminal;
3. envia o texto pelo socket.

9. Funcionamento do servidor

O servidor possui a ordem dos cinco desserializadores.

Para cada mensagem:

1. recebe o texto;
2. desserializa;
3. cria novamente o objeto `Mensagem`;
4. imprime nome, CPF, idade e mensagem.

10. Resultado

O servidor deve apresentar:

[1/5] FORMATO RECEBIDO: CSV
[2/5] FORMATO RECEBIDO: JSON
[3/5] FORMATO RECEBIDO: XML
[4/5] FORMATO RECEBIDO: YAML
[5/5] FORMATO RECEBIDO: TOML

Em todos os cinco casos, os valores recuperados devem ser iguais.

11. Capturas de tela

Figura 1 — Estrutura do projeto:

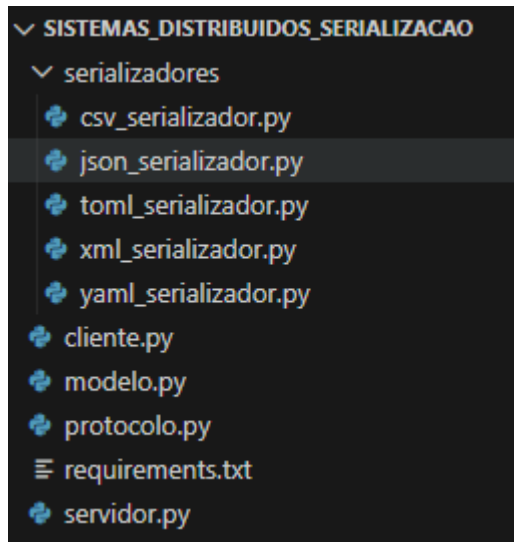


Figura 2 — Servidor aguardando conexão.

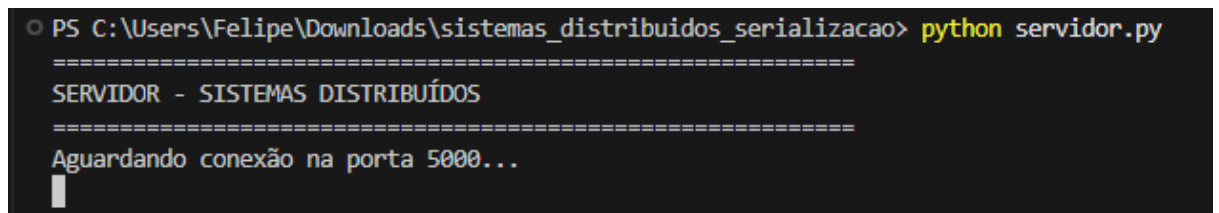


Figura 3 — Cliente enviando as cinco serializações.

```
PS C:\Users\Felipe\Downloads\sistemas_distribuidos_serializacao> python cliente.py
=====
CLIENTE - SISTEMAS DISTRIBUÍDOS
=====
Conectando em 127.0.0.1:5000...
Conexão estabelecida.

[1/5] Enviando em CSV...
Conteúdo serializado:
nome,cpf,idade,mensagem
Fulano da Silva,10326709722,45,segue comprovante de endereço

-----

[2/5] Enviando em JSON...
Conteúdo serializado:
{
  "nome": "Fulano da Silva",
  "cpf": "10326709722",
  "idade": 45,
  "mensagem": "segue comprovante de endereço"
}

-----

[3/5] Enviando em XML...
Conteúdo serializado:
<mensagem><nome>Fulano da Silva</nome><cpf>10326709722</cpf><idade>45</idade><mensagem>segue comprovante
de endereço</mensagem></mensagem>

-----

[4/5] Enviando em YAML...
Conteúdo serializado:
nome: Fulano da Silva
cpf: '10326709722'
idade: 45
mensagem: segue comprovante de endereço

-----

[5/5] Enviando em TOML...
Conteúdo serializado:
nome = "Fulano da Silva"
cpf = "10326709722"
idade = 45
mensagem = "segue comprovante de endereço"

-----

As 5 mensagens foram enviadas com sucesso.
Conexão encerrada.
```

Figura 4 — Servidor recebendo as mensagens.

```
PS C:\Users\Felipe\Downloads\sistemas_distribuidos_serializacao> python servidor.py

=====
SERVIDOR - SISTEMAS DISTRIBUÍDOS
=====
Aguardando conexão na porta 5000...
Cliente conectado: 127.0.0.1:53551

=====
[1/5] FORMATO RECEBIDO: CSV
=====
Nome      : Fulano da Silva
CPF       : 10326709722
Idade     : 45
Mensagem  : segue comprovante de endereço

Conteúdo recebido:
nome,cpf,idade,mensagem
Fulano da Silva,10326709722,45,segue comprovante de endereço

=====
[2/5] FORMATO RECEBIDO: JSON
=====
Nome      : Fulano da Silva
CPF       : 10326709722
Idade     : 45
Mensagem  : segue comprovante de endereço

Conteúdo recebido:
{
  "nome": "Fulano da Silva",
  "cpf": "10326709722",
  "idade": 45,
  "mensagem": "segue comprovante de endereço"
}

=====
[3/5] FORMATO RECEBIDO: XML
=====
Nome      : Fulano da Silva
CPF       : 10326709722
Idade     : 45
Mensagem  : segue comprovante de endereço

Conteúdo recebido:
<mensagem><nome>Fulano da Silva</nome><cpf>10326709722</cpf><idade>45</idade><mensagem>segue comprovante
de endereço</mensagem></mensagem>

=====
[4/5] FORMATO RECEBIDO: YAML
=====
Nome      : Fulano da Silva
CPF       : 10326709722
Idade     : 45
Mensagem  : segue comprovante de endereço

Conteúdo recebido:
nome: Fulano da Silva
cpf: '10326709722'
idade: 45
mensagem: segue comprovante de endereço
```



```
=====
[5/5] FORMATO RECEBIDO: TOML
=====
Nome      : Fulano da Silva
CPF       : 10326709722
Idade     : 45
Mensagem  : segue comprovante de endereço

Conteúdo recebido:
nome = "Fulano da Silva"
cpf = "10326709722"
idade = 45
mensagem = "segue comprovante de endereço"

=====
As 5 mensagens foram recebidas e processadas.
=====
```

12. Conclusão

A atividade permitiu colocar em prática conceitos de Sistemas Distribuídos por meio da comunicação entre cliente e servidor utilizando sockets TCP.

Também foi possível observar a diferença entre os formatos de serialização. A mesma informação foi enviada cinco vezes, em CSV, JSON, XML, YAML e TOML, e em todas as situações o servidor conseguiu recuperar os quatro campos originais.

A organização em módulos separados deixou o código mais simples de estudar, testar e apresentar.